

# Report 2: Hyperparameter Tuning Analysis

---

**Document Classification:** Technical Research Report **Date:** February 17, 2026 **Scope:** Analysis of how hyperparameter changes could affect VSP-LLM performance on real-world data  
**Current Performance Baseline:** Mean WER 67.0%, Corpus WER 125.5% on english\_1k (860 segments)

---

## 1. Executive Summary

---

The VSP-LLM decode pipeline has **9 tunable generation parameters** and several data/architecture-level knobs. This report analyzes each parameter's expected impact on the observed failure modes: hallucination (20.6% of segments), excessive insertions (corpus WER 125.5%), short-segment failure (mean WER 128.5% for 1-5 word segments), and named entity errors (NEA F1 38.8%).

**Key finding:** The most impactful parameter changes are (1) increasing the length penalty from 0.0 to 0.5-1.0 to directly combat the hallucination/insertion problem, and (2) increasing the minimum segment duration to 7+ seconds to exploit the model's context-dependent architecture. Together, these two changes alone could plausibly reduce the mean WER by 10-20 percentage points — though the fundamental domain gap would remain.

Hyperparameter tuning is a **mitigation strategy**, not a solution. The core problem is domain mismatch between LRS3 training data and real-world YouTube content.

---

## 2. Current Configuration

---

The model is deployed with the paper's exact recommended settings:

```
# From: VSP-LLM/src/conf/s2s_decode.yaml
generation:
  beam: 20                # Beam search width
  lenpen: 0.0             # Length penalty (0 = disabled)
  max_len_a: 2.0          # Dynamic max_len multiplier
  max_len_b: 200          # Dynamic max_len offset
  max_len: 2048           # Hard cap
  no_repeat_ngram_size: 3
  repetition_penalty: 1.2
  do_sample: false        # Deterministic beam search
  temperature: 1.0
  top_p: 0.9
```

**Source:** Paper Section 4.2: "For decoding, we use a beam search with a beam width of 20 and a length penalty of 0."

### 3. Parameter-by-Parameter Analysis

#### 3.1 Length Penalty ( `lenpen` ) — HIGHEST IMPACT

**Current value:** 0.0 (disabled) **What it does:** Divides the sequence log-probability by `lengthlenpen` during beam search. Values > 0 favor longer sequences; values < 0 penalize them. **Formula:** `score = log_prob(sequence) / lengthlenpen`

##### Why This Is the Most Important Parameter

Our corpus-level WER is **125.5%** — the model generates more error words than reference words. The dominant error type is **insertion**: the model generates extra words that don't exist in the reference. With `lenpen=0.0`, there is **zero penalty for generating longer sequences**, so the model is free to produce verbose, hallucinated text.

##### Evidence from the Data

Examining hallucinated segments (WER ≥ 100%):

| Reference                                                                   | Hypothesis                                                                                      | Ref Words | Hyp Words | Ratio |
|-----------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------|-----------|-----------|-------|
| "let's see a half a cup of"                                                 | "i have to say thank you"                                                                       | 7         | 6         | 0.86  |
| "has now carried you home"                                                  | "i dare you to hear the new home"                                                               | 5         | 8         | 1.60  |
| "it'll work"                                                                | "but"                                                                                           | 2         | 1         | 0.50  |
| "usually after lunch i'll get kind of sleepy so i'll have some iced coffee" | "you should have some rest and stay in bed and sleep and eat some cookies and have some coffee" | 14        | 18        | 1.29  |

The model frequently generates text of a different length than the reference, especially longer. A length penalty would constrain this.

##### Recommended Experiments

| <code>lenpen</code> | Expected Effect                                  | Risk                |
|---------------------|--------------------------------------------------|---------------------|
| 0.0<br>(current)    | No length preference — model hallucinates freely | High insertion rate |

| lenpen | Expected Effect                                                                       | Risk                                  |
|--------|---------------------------------------------------------------------------------------|---------------------------------------|
| 0.5    | Mild preference for shorter sequences — reduces unnecessary insertions                | May truncate some valid long outputs  |
| 1.0    | Standard length normalization (used by many NMT systems) — strong insertion reduction | May under-generate on longer segments |
| -0.5   | Penalizes long sequences — aggressive anti-hallucination                              | Will truncate valid outputs too       |

**Recommendation:** Test `lenpen=0.5` and `lenpen=1.0`. Based on machine translation literature (Koehn & Knowles, 2017; Wu et al., 2016 — "Google's NMT System"), `lenpen=0.6-1.0` is standard for beam search decoding and typically reduces degenerate outputs.

**Estimated impact:** Reducing insertions could lower corpus WER by 15-30 absolute percentage points (from 125.5% toward 95-110%). This primarily affects the 20.6% of segments that are catastrophically hallucinated.

### 3.2 Beam Width ( `beam` ) — MODERATE IMPACT

**Current value:** 20 **What it does:** Number of candidate sequences maintained during beam search. Higher values explore more hypotheses but increase memory usage and computation time.

#### Analysis

A beam width of 20 is unusually high. Common values in the literature:

| System                         | Beam Width | Domain          |
|--------------------------------|------------|-----------------|
| Google NMT (Wu et al., 2016)   | 8-12       | Translation     |
| OpenAI Whisper                 | 5          | ASR             |
| ESPnet (Watanabe et al., 2018) | 10-20      | ASR             |
| VSP-LLM (paper)                | 20         | Lip reading     |
| Typical LLM inference          | 1-4        | Text generation |

Large beam widths can cause **beam search degradation** — a known phenomenon where increasing the beam width paradoxically worsens output quality because the model finds higher-probability but less natural sequences (Koehn & Knowles, 2017; Murray & Chiang, 2018). This is especially problematic when `lenpen=0.0`, because longer sequences accumulate more probability mass.

## Recommended Experiments

| beam         | Expected Effect                                                       |
|--------------|-----------------------------------------------------------------------|
| 5            | Faster decode, may reduce degenerate modes. Standard for ASR.         |
| 10           | Balanced exploration vs. speed                                        |
| 20 (current) | Maximum exploration — but may find degenerate solutions               |
| 1 (greedy)   | Fastest; avoids beam search pathologies entirely; baseline comparison |

**Recommendation:** Test `beam=5` and `beam=10` alongside `lenpen=0.5`. Reducing beam width from 20 to 5 would also reduce GPU memory usage by approximately 4x during generation, enabling longer segments.

**Estimated impact:** Beam width changes alone are unlikely to dramatically improve WER ( $\pm 2$ -5 points), but combined with proper length penalty, they can reduce degenerate outputs.

---

## 3.3 Repetition Penalty ( `repetition_penalty` ) — LOW-MODERATE IMPACT

**Current value:** 1.2 **What it does:** Divides the logit of a previously-generated token by this factor before the next softmax. Values  $> 1.0$  discourage repetition; 1.0 = disabled. From CTRL paper (Keskar et al., 2019).

### Analysis

The current value of 1.2 is reasonable and within the standard range (1.1-1.5). Combined with `no_repeat_ngram_size=3`, it prevents most repetitive degeneration.

However, for lip reading, moderate repetition is **natural** — speakers often repeat words ("the the", "you know you know"). A penalty of 1.2 may suppress valid repetitions.

## Recommended Experiments

| Value         | Expected Effect                                         |
|---------------|---------------------------------------------------------|
| 1.0 (off)     | Allows natural repetition; risk of degenerate loops     |
| 1.1           | Mild penalty; more natural for conversational speech    |
| 1.2 (current) | Standard; prevents most loops                           |
| 1.5           | Aggressive; may force vocabulary diversity artificially |

**Estimated impact:**  $\pm 1$ -3 WER points. Not a primary lever.

---

### 3.4 No-Repeat N-gram Size ( `no_repeat_ngram_size` ) — LOW IMPACT

**Current value:** 3 **What it does:** Prevents the model from generating any trigram that already appeared in the output. Hard constraint (probability set to 0).

#### Analysis

This is a safety valve against degenerate repetition loops. With value 3, the model can never output "the quick brown the quick brown" because "the quick brown" would repeat.

A value of 3 is reasonable. Reducing to 2 would be too aggressive (many natural bigrams repeat: "of the", "to the"). Increasing to 4 or 5 would provide less protection.

**Recommendation:** Keep at 3. Not a significant lever for our failure modes.

---

### 3.5 Sampling Parameters ( `do_sample` , `temperature` , `top_p` ) — EXPLORATORY

**Current values:** `do_sample=false` , `temperature=1.0` , `top_p=0.9`

Since `do_sample=false` , `temperature` and `top_p` are **inactive**. The model uses pure beam search.

#### Analysis: Should We Enable Sampling?

Sampling introduces stochasticity — the model randomly selects from its probability distribution rather than always choosing the most likely token. This can increase diversity but reduces reproducibility.

For lip reading, sampling is generally **not recommended** because: 1. We want the most likely interpretation, not creative diversity 2. Sampling would make results non-reproducible 3. Beam search already explores multiple candidates

However, **nucleus sampling with low temperature** could be interesting for a specific use case: generating multiple diverse hypotheses for later aggregation (see Report 5).

| Configuration                                                                        | Use Case                                                 |
|--------------------------------------------------------------------------------------|----------------------------------------------------------|
| <code>do_sample=false</code> (current)                                               | Deterministic, reproducible, best single hypothesis      |
| <code>do_sample=true</code> , <code>temperature=0.3</code> , <code>top_p=0.9</code>  | Slightly diversified — might avoid degenerate beam modes |
| <code>do_sample=true</code> , <code>temperature=0.7</code> , <code>top_p=0.95</code> | High diversity — useful only for hypothesis aggregation  |

**Recommendation:** Keep `do_sample=false` for production. Consider sampling only for the aggregation approach in Report 5.

---

### 3.6 Dynamic Max Length ( `max_len_a` , `max_len_b` , `max_len` ) — MODERATE IMPACT

**Current values:** `max_len_a=2.0` , `max_len_b=200` , `max_len=2048` **Formula:**  
`dynamic_max_len = max_len_a * src_clusters + max_len_b`

#### Analysis

For a typical segment with ~50 visual clusters after deduplication: - `dynamic_max_len = 2.0 * 50 + 200 = 300 tokens` - At ~1.3 tokens/word, this allows ~230 words of output

For a 5-second segment at 25 fps = 125 frames, after deduplication (~50% reduction) = ~60 clusters: - `dynamic_max_len = 2.0 * 60 + 200 = 320 tokens` (~250 words)

A 5-second clip typically contains ~15 spoken words. The model is allowed to generate **16x more text than the speaker actually said**. This directly enables hallucination — the model has room to generate 250 words when only 15 were spoken.

#### Recommended Changes

| Parameter              | Current | Proposed | Rationale                |
|------------------------|---------|----------|--------------------------|
| <code>max_len_a</code> | 2.0     | 1.0      | Tighter proportional cap |
| <code>max_len_b</code> | 200     | 50       | Smaller buffer           |
| <code>max_len</code>   | 2048    | 512      | Lower hard cap           |

With `max_len_a=1.0` , `max_len_b=50` : for 60 clusters → max 110 tokens (~85 words). Still generous but prevents extreme hallucination.

**Estimated impact:** Reduces catastrophic hallucinations where the model generates extremely long fabricated text. Affects primarily the tail of the WER distribution ( $\geq 100\%$  segments). Expected to lower corpus WER by 5-15 points.

**Risk:** Setting too low will truncate valid long outputs. Needs empirical testing.

---

### 3.7 Video Segment Length — HIGHEST IMPACT (data-level)

**Not a decode hyperparameter** but the most impactful variable based on our data.

## Evidence from the Paper

Paper Figure 4 — VSP-LLM WER by video length (LRS3):

| Video Length | WER          |
|--------------|--------------|
| 0-2 seconds  | 34.7%        |
| 2-4 seconds  | 22.5%        |
| 4-6 seconds  | 17.0%        |
| 6+ seconds   | <b>12.9%</b> |

The model's WER improves by **63%** (34.7% → 12.9%) as video length increases from 0-2s to 6+s on LRS3.

## Evidence from Our Data

| Ref Word Count | Approx Duration | Mean WER | % of Segments |
|----------------|-----------------|----------|---------------|
| 1-5 words      | ~1-3s           | 128.5%   | ~8%           |
| 6-10 words     | ~3-5s           | 72.3%    | ~20%          |
| 11-20 words    | ~5-10s          | ~60%     | ~40%          |
| 21+ words      | ~10-12s         | 56.6%    | ~32%          |

## Recommendation

The current pipeline segments videos at a maximum of 12 seconds. Consider:

1. **Increase minimum segment length to 5 seconds** — discard or merge short segments
2. **Increase maximum segment length to 20-30 seconds** — allows more context (requires GPU memory management)
3. **Overlap segments more aggressively** — 50% overlap instead of current settings, then merge predictions

**Estimated impact:** Filtering to segments  $\geq 5$  seconds: mean WER drops from 67% to ~58% (retaining ~47% of segments). Filtering to  $\geq 7$  seconds: mean WER ~54% (retaining ~35% of segments). This is the single most impactful change available without modifying the model.

**Trade-off:** Longer segments require more GPU memory. With beam=20 and max\_len=2048, very long segments may exceed 16GB VRAM. Reducing beam width to 5-10 mitigates this.

---

### 3.8 K-means Cluster Count — LOW IMPACT AT INFERENCE

**Current value:** 200 clusters **What it does:** Controls the granularity of visual speech unit deduplication. More clusters = finer-grained visual features = longer sequences.

#### Paper Evidence (Table 3)

| Clusters | Avg BLEU (VST) | Sequence Length | FLOPs |
|----------|----------------|-----------------|-------|
| None     | 14.6           | 1.00x           | 62.4P |
| 2000     | 14.4           | 0.70x           | 53.8P |
| 200      | 14.5           | 0.53x           | 45.6P |
| 50       | 14.3           | 0.45x           | 41.0P |

The paper shows that cluster count has **minimal impact on accuracy** (BLEU varies from 14.3 to 14.6) but significant impact on computational cost. The current value of 200 is the paper's recommended balance.

#### Real-World Consideration

Changing the cluster count requires **re-running the entire clustering stage** (K-means on all features), which is computationally expensive. The expected accuracy change is negligible based on the paper's ablation.

**Recommendation:** Keep at 200. Not a useful lever for improving real-world WER.

---

### 3.9 Model-Level Parameters (Not Tunable at Inference Time)

These require retraining or fine-tuning the model:

| Parameter        | Current                       | Notes                                                                       |
|------------------|-------------------------------|-----------------------------------------------------------------------------|
| LoRA rank        | 16                            | Higher rank (32, 64) = more trainable params, potentially better adaptation |
| Encoder freezing | Frozen (FT version unfreezes) | Paper shows FT improves WER from 26.7% to 25.4%                             |
| Training data    | 433h LRS3                     | Domain adaptation with target data would be the most impactful change       |
| LLM backbone     | LLaMA-2-7B                    | Larger models (13B, 70B) have stronger language priors but higher cost      |
| Visual encoder   | AV-HuBERT Large               |                                                                             |



| Parameter | Current | Notes                                                    |
|-----------|---------|----------------------------------------------------------|
|           |         | Pre-trained on LRS3 + VoxCeleb2; not readily replaceable |

**These are not viable for short-term improvement** but are mentioned for completeness.

## 4. Recommended Tuning Strategy

### 4.1 Phase 1: Quick Wins (No Retraining)

Run the following configurations on the full english\_1k dataset and compare:

| Experiment               | beam | lenpen | rep_pen | max_len_a | max_len_b | Expected Effect                |
|--------------------------|------|--------|---------|-----------|-----------|--------------------------------|
| Baseline                 | 20   | 0.0    | 1.2     | 2.0       | 200       | Current: WER 67%               |
| A: Length penalty        | 20   | 0.5    | 1.2     | 2.0       | 200       | Reduce insertions              |
| B: Strong length penalty | 20   | 1.0    | 1.2     | 2.0       | 200       | Aggressively reduce insertions |
| C: Smaller beam          | 5    | 0.5    | 1.2     | 2.0       | 200       | Faster + anti-degenerate       |
| D: Tighter max length    | 20   | 0.5    | 1.2     | 1.0       | 50        | Cap hallucination length       |
| E: Combined              | 5    | 1.0    | 1.1     | 1.0       | 50        | All mitigations together       |
| F: Greedy baseline       | 1    | 0.0    | 1.0     | 2.0       | 200       | Sanity check                   |

Each experiment takes approximately the same time as the current decode run.

### 4.2 Phase 2: Segment Length Optimization

Re-segment the input videos with different parameters:

| Experiment           | Min Length | Max Length | Overlap  | Expected Effect     |
|----------------------|------------|------------|----------|---------------------|
| Current              | ~1s        | 12s        | Variable | Baseline            |
| G: No short segments | 5s         | 12s        | Same     | Drop worst segments |

| Experiment         | Min Length | Max Length | Overlap | Expected Effect          |
|--------------------|------------|------------|---------|--------------------------|
| H: Longer segments | 5s         | 20s        | 50%     | More context per segment |
| I: Very long       | 7s         | 30s        | 50%     | Maximum context          |

### 4.3 Phase 3: Combined Best Settings

Take the best settings from Phase 1 + Phase 2 and combine. Expected best case: **WER reduction from 67% to ~45-50%** (still far from the paper's 25.4% on LRS3, due to the irreducible domain gap).

## 5. Expected Outcomes and Limitations

### 5.1 Realistic Expectations

| Optimization                | Expected WER Change | Confidence |
|-----------------------------|---------------------|------------|
| Length penalty (0.5-1.0)    | -5 to -15 points    | High       |
| Segment length $\geq$ 5s    | -5 to -10 points    | High       |
| Beam width reduction (5-10) | -1 to -3 points     | Medium     |
| Max length tightening       | -2 to -5 points     | Medium     |
| Repetition penalty tuning   | $\pm 1$ point       | Low        |
| All combined                | -15 to -25 points   | Medium     |

**Best realistic outcome with hyperparameter tuning alone:** WER ~45-55% (down from 67%).

### 5.2 The Irreducible Domain Gap

Even with optimal hyperparameters, the model will not approach its LRS3 benchmark (25.4% WER) on real-world YouTube data because:

1. **Visual encoder limitations:** AV-HuBERT was trained on LRS3/VoxCeleb2 — frontal, well-lit faces. It extracts poor features from side profiles, poor lighting, and occluded mouths.
2. **LLM vocabulary bias:** LLaMA-2 was fine-tuned (LoRA) on LRS3 text — TED talk language. When visual features are ambiguous, it defaults to TED-talk-like text, not YouTube-style casual speech.

3. **Homophone overload:** On LRS3, the vocabulary is constrained enough that context resolves most ambiguity. On YouTube, the vocabulary is open and context often insufficient.

To close the domain gap, model retraining on target-domain data is required.

---

## 6. Implementation Guide

---

### 6.1 How to Run Parameter Experiments

Modify `VSP-LLM/src/conf/s2s_decode.yaml` or pass overrides via command line:

```
# Example: Experiment A (length penalty 0.5)
CUDA_VISIBLE_DEVICES=0 python -B ${MODEL_SRC}/vsp_llm_decode.py \
  --config-dir ${MODEL_SRC}/conf \
  --config-name s2s_decode \
  generation.beam=20 \
  generation.lenpen=0.5 \
  # ... other args unchanged
```

Or modify the YAML directly:

```
# Experiment E: Combined best settings
generation:
  beam: 5
  lenpen: 1.0
  max_len_a: 1.0
  max_len_b: 50
  max_len: 512
  no_repeat_ngram_size: 3
  repetition_penalty: 1.1
  do_sample: false
  temperature: 1.0
  top_p: 0.9
```

### 6.2 Automated Sweep Script (Conceptual)

```
#!/bin/bash
# hyperparameter_sweep.sh
for lenpen in 0.0 0.5 1.0; do
  for beam in 5 10 20; do
    echo "Running: beam=$beam lenpen=$lenpen"
    # Run decode with these params
    # Each run produces a unique hypo-{hash}.json due to config hashing
    python -B vsp_llm_decode.py \
```

```
--config-dir conf --config-name s2s_decode \  
generation.beam=$beam generation.lenpen=$lenpen \  
# ... other fixed args  
# Generate report  
python make_report.py --jsonl decode/vsr/en/hypo-*.json --out_dir results/sweep_b${beam}  
done  
done
```

Note: Each full decode run on 860 segments takes approximately 2-4 hours on a single GPU. A 6-experiment sweep requires 12-24 hours of GPU time.

---

## 7. References

---

1. Yeo et al. (2024). "Where Visual Speech Meets Language: VSP-LLM Framework." arXiv: 2402.15151.
2. Wu et al. (2016). "Google's Neural Machine Translation System." arXiv:1609.08144. — Length penalty and beam search best practices.
3. Koehn & Knowles (2017). "Six Challenges for Neural Machine Translation." arXiv: 1706.03872. — Beam search degradation phenomenon.
4. Murray & Chiang (2018). "Correcting Length Bias in Neural Machine Translation." arXiv: 1808.10006. — Length normalization analysis.
5. Keskar et al. (2019). "CTRL: A Conditional Transformer Language Model." arXiv: 1909.05858. — Repetition penalty.
6. Holtzman et al. (2020). "The Curious Case of Neural Text Degeneration." ICLR. — Nucleus sampling vs. beam search.
7. Afouras et al. (2018). "LRS3-TED: A Large-Scale Dataset for Visual Speech Recognition." arXiv:1809.00496.